

Programación de Servicios y Procesos

Programación multiproceso

Tema 1

1. Introducción: Aplicaciones, Ejecutables y Procesos

Programa Conjunto de **instrucciones** que ejecutadas en un ordenador **realizarán una tarea o ayudarán a realizarla**.

Aplicación Tipo de programa informático, diseñado como herramienta para resolver de manera automática **un problema específico** del usuario.

Ejecutable Fichero que contiene el **código binario o interpretado** que será ejecutado en un ordenador.

Proceso Programa en ejecución que existe **mientras se esté ejecutando una aplicación**. La ejecución de una aplicación puede implicar **varios procesos**.

Servicio Proceso, normalmente cargado **durante el arranque** del SO, que queda a la espera de que otro le pida que realice una tarea.

1.1. Ejecutables. Tipos.

Binarios Conjunto de **instrucciones ejecutadas por el procesador** del ordenador obtenido al **compilar el código fuente** de un programa. Solo se ejecutará en equipos cuya plataforma **sea compatible** con aquella para la que ha sido compilado.

Interpretados Código que se trata como un ejecutable pero **no es código binario**. Formado por códigos de operación que el intérprete traducirá al lenguaje máquina del procesador. Más susceptible de ser **independiente de la máquina** en la que se haya compilado.

Scripts Ficheros **no compilados** que contienen instrucciones que serán ejecutados una tras otra por el intérprete.

Librerías Conjunto de funciones que permiten dar **modularidad y reusabilidad a los programas**. Código ejecutable que es ejecutado por todos los programas que invoquen las funciones que contienen.

2. Gestión de procesos

Los **tipos de proceso** pueden clasificarse en:

- **Por lotes** Formados por una serie de tareas de las que el usuario solo está interesado en **el resultado final**.
- **Interactivos** Tareas en las que el proceso **interactúa continuamente con el usuario** y actúa de acuerdo a las acciones que realiza o los datos que suministra.
- **Tiempo real** Tareas en las que es crítico el **tiempo de respuesta** del sistema.

2.1. Estados de un proceso

1. Los procesos nuevos **entran en la cola** de procesos activos.
2. Los procesos van **avanzando en la cola** de procesos activos, hasta que les toca el turno para **recibir el uso de la CPU**.
3. El **SO concede el uso de la CPU** a cada proceso durante un tiempo determinado y equitativo llamado **quantum**. Un proceso que consume su quantum es pausado y **enviado al final de la cola**.
4. Si un proceso finaliza, **sale del sistema** de gestión.

Cuando un proceso necesita datos de un archivo o una entrada de datos, o tiene que emitir datos, entra en una **operación de entrada/salida** durante la que está **bloqueado**: otros procesos pueden pasar a la CPU y ejecutar sus instrucciones mientras este está bloqueado. Volverá a la cola de procesos cuando la operación E/S finalice.

2.2. Planificación de procesos por el Sistema Operativo

2.2.1. Cargador

Componente del SO encargado de **crear los procesos**. Cuando se inicia un proceso:

1. Carga el proceso en memoria principal, copiando a un espacio en la RAM las **instrucciones del fichero ejecutable**, las constantes, y deja un espacio para los **datos y la pila**. Un proceso **no podrá hacer referencia** a direcciones que se encuentren **fuera de su espacio de memoria**.

2. Crea una estructura de información llamada **PCB**, única para cada proceso, que permite controlarlo. Contiene:

- **PID** del proceso.
- **Estado** actual.
- Espacio de **direcciones de memoria**.
- **Planificación**: Prioridad, quantum, estadísticas...
- Información para el **cambio de contexto**.
- **Recursos** utilizados.

2.2.2. Planificador

Componente del SO encargado de **tomar las decisiones relacionadas con la ejecución de los procesos**, mediante una política determinada **algoritmo de planificación**:

- **Round-Robin** Favorece la ejecución de **procesos interactivos**. Cada proceso puede ejecutar sus instrucciones en la CPU durante **un quantum**. Si no le da tiempo a finalizar, se coloca al final de la cola de procesos, donde **espera hasta que vuelva a ser su turno**.
- **Por prioridad** Se asignan **prioridades** a los distintos procesos y la ejecución de estos se hace **de acuerdo a la prioridad asignada**.
- **Múltiples colas** Combinación de los dos anteriores, implementado en los SO actuales. **Todos los procesos de una misma prioridad estarán en la misma cola, que será gestionada mediante RR**. Los procesos de colas de **inferior prioridad** no se ejecutarán hasta que se hayan vaciado las colas de **mayor prioridad**.

La planificación busca conciliar:

- **Equidad** Todos los procesos deben poder ejecutarse.
- **Eficacia** Mantener ocupada la CPU un 100 % del tiempo.
- **Tiempo de respuesta** Minimizar el tiempo de respuesta al usuario.
- **Tiempo de regreso** Minimizar el tiempo que deben esperar los usuarios de procesos por lotes.
- **Rendimiento** Maximizar el número de tareas procesadas por hora.

2.3. Cambio de contexto en la CPU

Un proceso es una **unidad de trabajo completa**, formado por al menos **un hilo de ejecución**. El sistema operativo es el encargado de gestionar los procesos en ejecución de forma eficiente, intentando evitar que haya conflictos. Se asocia a cada proceso un **conjunto de información (PCB)** y **unos mecanismos de protección**.

Estado de la CPU Conjunto de espacios de memoria (**registros**) en los que se almacenan temporalmente la información que necesita la instrucción que está procesando la CPU. La CPU realiza un cambio de contexto **cada vez que cambia la ejecución de un proceso a otro**, momento en el que guarda el estado actual y restaura el estado del proceso que va a pasar a ejecutar.

Destacan:

- **Contador de programa** Almacena la dirección de la siguiente instrucción a ejecutar. Permitirá continuar en cada proceso por la instrucción donde fue dejado.
- **Puntero a pila** Apunta a la parte superior de la pila del proceso en ejecución.

Realizar cambios de contexto entre hilos de un mismo proceso es **más rápido** y **menos costoso** que el cambio de contexto entre procesos completos.

2.4. Creación de procesos

Las clases necesarias para la creación de procesos en Java son:

- **java.lang.Process** Proporciona los objetos **Process** con los que se puede controlar los procesos creados.
- **java.lang.Runtime** Clase que permite lanzar la ejecución de un programa en el sistema.

2.5. Comandos para la gestión de procesos

2.5.1. Windows

- **tasklist** Lista los procesos presentes.
- **taskkill** Mata procesos. La opción /PID permite especificar el **identificador** del proceso.

2.5.2. Linux

- **ps** Lista los procesos presentes. La opción **aux** muestra todos, **independientemente del usuario** que los lance.
- **pstree** Listado de procesos en **forma de árbol**, mostrando qué procesos han creado otros.
- **kill** Manda señales a los procesos. El comando **kill -9 PID** **matará el proceso**.
- **killall** Mata procesos por su nombre.
- **nice** Cambia la prioridad de un proceso con la sintaxis **nice -n X** comando, donde **X** es la prioridad. La prioridad por defecto es 0 y se sitúa entre **-20 (más alta)** y **19 (más baja)**.

3. Programación concurrente

Programación concurrente Proporciona **mecanismos de comunicación y sincronización** entre procesos que se ejecutan **de forma simultánea** en un sistema informático. Permite definir qué instrucciones de los procesos se pueden ejecutar de forma simultánea con las de otros procesos, y cuales deben ser **sincronizadas** para que los resultados sean correctos.

Dos procesos son **concurrentes** cuando **la primera instrucción** de un proceso se ejecuta **después de la primera y antes de la última** de otro proceso.

Las **principales razones** para usar una estructura concurrente son:

- Optimizar la **utilización de recursos**.
- Proporcionar **interactividad** a los usuarios.
- Mejorar la **disponibilidad**.
- Conseguir un diseño más **comprensible y mantenible**.
- **Aumentar la protección** al aislar cada tarea en un proceso.

La concurrencia aporta **gran claridad** sobre lo que cada proceso hace y cuando, **reduce el tiempo** de ejecución, y permite un **mejor modelado previo** del comportamiento del programa.

Los tipos de interacción entre procesos son:

- **Independientes** Solo interfieren en el uso de la CPU.
- **Cooperantes** Un proceso genera la información o proporciona un servicio que otro necesita.
- **Competidores** Procesos que necesitan usar los mismos recursos de forma exclusiva.

3.1. Condiciones de competencia

Un proceso entra en condición de competencia con otro cuando ambos necesitan el mismo recurso, sea de forma exclusiva o no.

Región de exclusión mutua / región crítica Conjunto de instrucciones en las que un **proceso utiliza un recurso** y se deben ejecutar de forma exclusiva con respecto a otros procesos competidores por ese mismo recurso. Las instrucciones se ejecutarán de **forma indivisible y exclusiva con respecto a otros procesos que accedan al mismo recurso**. La definición se expande en la [sección 5.1](#).

Lock (bloqueo) Situación sobre un recurso cuando un **proceso que requiere un recurso ha obtenido su uso** en exclusión mutua.

Deadlock (interbloqueo) Situación cuando uno o más procesos **no pueden obtener nunca los recursos necesarios para continuar su tarea**, puesto que otro proceso ha bloqueado un recurso que otro necesita y viceversa.

4. Comunicación entre procesos

Primitiva de sincronización Operación proporcionada por los lenguajes de programación o el SO, de la cual **conocemos sus restricciones y objetos** pero no su implementación, que **facilita la interacción entre procesos**.

Las **interacciones entre procesos y el resto del sistema** son:

- **Sincronización** Un proceso puede **conocer el punto de ejecución** en el que se encuentra otro en este instante.
- **Exclusión mutua** Mientras un proceso accede a un recurso, **ningún otro proceso** puede acceder a él.

- **Sincronización condicional** Solo se puede acceder a un recurso cuando se encuentra en un **determinado estado interno**.

4.1. Mecanismos básicos de comunicación

- **Intercambio de mensajes** mediante las primitivas `send` y `receive / wait`.
- **Recursos (o memoria) compartidos** mediante las primitivas `write` y `send`.

Para comunicar procesos dentro de una misma máquina, el intercambio de mensajes se puede realizar mediante **un buffer de memoria o un socket**. Un **socket** se utiliza para intercambiar información entre procesos en **distintas máquinas a través de la red**; un **buffer** crea un canal de comunicación entre dos procesos **utilizando la memoria principal del sistema**.

Una escritura **bloqueará el proceso** que intenta escribir hasta que el recurso esté preparado para poder escribir.

4.2. Tipos de comunicación

4.2.1. Según la capacidad del canal

- **Simplex** La comunicación se produce **en un solo sentido**. El **emisor es origen** del mensaje y el **receptor escucha el mensaje** al final del canal.
- **Dúplex** Pueden viajar mensajes en **ambos sentidos simultáneamente**.
- **Semidúplex** El mensaje puede viajar en ambos sentidos, pero **no al mismo tiempo**.

4.2.2. Según la sincronía

- **Síncrona** El emisor queda **bloqueado hasta que el receptor recibe el mensaje**.
- **Asíncrona** El emisor **continúa con su ejecución inmediatamente** después de emitir el mensaje.
- **Invocación remota** El proceso emisor queda **suspendido hasta que recibe la confirmación** de que el receptor ha recibido correctamente el mensaje.

4.2.3. Según el comportamiento de los interlocutores

- **Simétrica** Todos los procesos pueden enviar y recibir información.
- **Asimétrica** Solo un proceso actúa de emisor.

5. Sincronización entre procesos

5.1. Regiones críticas

Como se definió en la [sección 3.1](#), **conjunto de instrucciones en las que un proceso accede a un recurso compartido**. Las instrucciones se ejecutarán de forma **indivisible y exclusiva** con respecto a otros procesos que accedan al mismo recurso compartido.

Se protegerán con secciones críticas solo **aquellas instrucciones que acceden a un recurso compartido**, y las instrucciones **serán** las mínimas e imprescindibles que deben ser ejecutadas de forma atómica.

Un único proceso entra en la sección crítica, y el resto de procesos esperan a que este salga, porque encontrarán el **recurso bloqueado**. Al final de cada sección crítica, **el recurso debe ser liberado** para que otros procesos puedan utilizarlo.

Cualquier actualización de datos en un recurso compartido necesitará establecer una región crítica con las siguientes instrucciones:

- Leer el dato que se quiera actualizar.
- Realizar el cálculo de actualización.
- Escribir el dato actualizado.

Se pueden definir **tantas secciones críticas** como sean necesarias.

5.1.1. Categoría de proceso cliente-suministrador

Cliente Proceso que **requiere o solicita información o servicios** que proporciona otro proceso.

Suministrador Proceso que **suministra información o servicios a otros procesos**. La información es **perecedera** y desaparece cuando es consumida por un cliente.

Entre un cliente y un suministrador se establece **sincronismo** mediante intercambio de mensajes o a través de un recurso compartido.

Un cliente **no debe poder leer un dato** hasta que no haya sido **completamente** suministrado. El suministrador irá produciendo información que no podrá superar un volumen de **tamaño máximo establecido**.

5.2. Semáforos

Semáforo Componente de **bajo nivel de abstracción** que permite arbitrar los accesos a un recurso compartido en un recurso compartido en un entorno de programación concurrente. Pueden ser **binarios o generales**. Proporcionan una **gran capacidad funcional**, pero su **bajo nivel de abstracción** los hace peligrosos de manejar.

El semáforo es un **tipo dato instanciable** que tomará un determinado conjunto de valores y realizará un determinado conjunto de operaciones. Tendrá asociada una **lista de procesos** que, suspendidos, se encuentran a la espera de entrar en el mismo.

Permite **dos operaciones** seguras:

- **wait** Si el semáforo está abierto, **reduce en uno su valor**. Si está cerrado, el proceso que lo ejecuta **se suspende** y se encola en la lista de procesos.
- **signal** Si hay algún proceso en la lista de procesos, **activa uno de ellos** para que ejecute la sentencia que sigue al **wait** que lo suspendió. Si no hay procesos en espera, **incrementa en 1 el valor** del semáforo.

El procedimiento de uso es:

1. Un proceso padre **crea e inicializa** el semáforo.
2. El proceso padre creará **el resto de procesos hijo** pasándoles el semáforo creado.
3. Cada proceso hijo hará uso de **wait** para **consultar si puede acceder** a la sección crítica, y **signal** para indicar que **abandona su sección** y otro proceso puede entrar.

5.2.1. Ejemplo de semáforo en Java

```
1 import java.util.concurrent.Semaphore;
2
3 public class EjemploSemaforo {
4
5     public static void main(String[] args) {
```

```

6      Semaphore semaphore = new Semaphore(3); // Numero de hilos simultaneos
7
8      for (int i = 0; i < 10; i++) {
9          new Thread(new Worker(semaphore)).start();
10     }
11 }
12 }
13
14 class Worker implements Runnable {
15     private final Semaphore semaphore;
16
17     Worker(Semaphore semaphore) {
18         this.semaphore = semaphore;
19     }
20
21     @Override
22     public void run() {
23         try {
24             semaphore.acquire(); // Equivalente a wait
25             System.out.println(Thread.currentThread().getName() + " esta trabajando.");
26             Thread.sleep((long) (Math.random() * 1000)); // Simulamos un proceso
27             System.out.println(Thread.currentThread().getName() + " ha terminado.");
28         } catch (InterruptedException e) {
29             e.printStackTrace();
30         } finally {
31             semaphore.release(); // Equivalente a signal
32         }
33     }
34 }

```

5.3. Monitor

Monitor Componente de **alto nivel de abstracción** destinado a **gestionar recursos que van a ser accedidos de forma concurrente**. Encierran en su interior los **recursos o variables compartidas** como componentes privados y garantizan el acceso a ellas en exclusión mutua.

La declaración de un monitor incluye:

- **Declaración** de las constantes, variables, procedimientos y funciones que son privados del monitor.
- Declaración de los **procedimientos y funciones** que el monitor expone y constituye la interfaz para el acceso de los procesos.
- **Cuerpo del monitor**, constituido por un bloque de código que se ejecuta al instanciar el monitor, e inicializa las variables y estructuras internas.
- **Lista de procesos** que son suspendidos al tratar de acceder al monitor.

Java **no implementa** una clase Monitor.

5.4. Cola de mensajes

Paso de mensajes Técnica empleada en programación concurrente para aportar **sincronía entre procesos y permitir la exclusión mutua**. No precisa de memoria compartida.

- **Asíncrono** El proceso que envía **no espera a que el mensaje sea recibido** y continúa su ejecución. Suele emplear colas, en las que se almacenan los mensajes a la espera de que un proceso los reciba.
- **Síncrono** El proceso que envía el mensaje **espera a que un proceso lo reciba** para continuar su ejecución.

6. Requisitos: Seguridad, vivacidad, eficiencia y reusabilidad

Todo programa concurrente debe satisfacer **dos tipos de propiedades**

- **Seguridad** Propiedades relativas a que en cada instante de la ejecución **no debe haberse producido** algo que haga entrar al programa en un estado erróneo.
 - Dos procesos no deben entrar simultáneamente en una **sección crítica**.
 - Se respetan las **condiciones de sincronismo**.
- **Vivacidad** Cada sentencia que se ejecute conduce a **un avance constructivo** para alcanzar el objetivo funcional del programa.
 - No deben producirse **bloqueos activos**.
 - **Aplazamiento indefinido (starvation)** Estado al que puede llegar un programa que aunque **potencialmente** pueda avanzar de forma constructiva, porque no se le ha asignado tiempo de procesador, o porque las condiciones de sincronización le perjudican.
 - **Interbloqueo** Mencionado en la [sección 3.1](#), procesos que no pueden obtener los recursos necesarios para completar su tarea.

6.1. Arquitecturas y patrones de diseño

Arquitectura del software / arquitectura lógica Diseño de más alto nivel de la estructura de un sistema. **Conjunto de patrones y abstracciones**

coherentes con base a las cuales se pueden **resolver problemas**. Indican la **estructura, funcionamiento e interacción** entre las partes del software.

Patrón de diseño Soluciones de diseño que son **válidas en distintos contextos** y han sido aplicadas con éxito en otras ocasiones.

La arquitectura es el resultado de **ensamblar un cierto número de elementos** para satisfacer la **mayor funcionalidad y requisitos** de desempeño de un sistema, así como otros requisitos como **confiabilidad, escalabilidad, portabilidad, disponibilidad, mantenibilidad, auditabilidad, flexibilidad e interacción**.

Las arquitecturas más conocidas son:

- **Monolítica** El software se estructura en **grupos funcionales** muy acoplados.
- **Cliente-servidor** El software reparte su carga de cómputo en dos partes: **consumir un servicio y proporcionar un servicio**.
- **Arquitectura de tres niveles** Especialización de la arquitectura cliente-servidor donde **la carga se divide en capas** con un reparto claro de funciones: **interfaz de usuario, cálculo y almacenamiento**. Una capa solamente tiene relación con la siguiente.

Otras arquitecturas menos conocidas son:

- **En pipeline** Modelar un proceso **comprendido por varias fases secuenciales**, siendo la entrada de cada una la salida de la anterior.
- **Entre pares** Cada elemento es igual a otro: **actúa simultáneamente como cliente y como servidor**.
- **En pizarra** Múltiples **elementos funcionales** (agentes) que estarán especializados en una **tarea concreta** y un **instrumento de control** (pizarra) donde los agentes leerán y escribirán resultados.
- **Orientada a servicios** Se diseñan servicios de aplicación basados en una definición formal **independientemente de la plataforma** y el lenguaje.
- **Dirigida por eventos** Centrada en la **producción, detección, consumo y reacción** de eventos.

7. Programación paralela y distribuida

Dos procesos se ejecutan de forma paralela **si las instrucciones de ambos se están ejecutando realmente de forma simultánea.**

La **programación paralela** se centra en **microprocesadores multinúcleo o en supercomputadores**. Los procesos pueden intercambiar datos mediante **memoria compartida o una red de interconexión propia.**

La **programación distribuida** se centra en **conjuntos de ordenadores heterogéneos interconectados entre sí**. El intercambio de datos y la sincronización se realizará mediante intercambios de mensajes.

Cada procesador tiene la tarea de resolver una porción del problema. El sistema se presenta como una unidad, ocultando la realidad de las partes que lo forman.

7.1. Conceptos básicos

7.1.1. Definiciones

Distribución Construcción de una aplicación **por partes**, donde cada parte tiene un **conjunto de responsabilidades dentro del sistema.**

Nudo de la red Uno o varios equipos que se comportan como una **unidad de asignación** integrada en el sistema.

Objeto distribuido Módulo de código con **plena autonomía** que se puede instanciar en **cualquier nudo** y a cuyos servicios pueden acceder clientes ubicados en **cualquier otro nudo.**

Componente Elemento de software que **encapsula una serie de funcionalidades**, puede ser utilizado en conjunto con otros para formar sistemas más complejos, y **no está ligado a ninguna aplicación.**

Transacciones Conjunto de actividades que **se ejecutan en diferentes nudos de una plataforma** distribuida para ejecutar una tarea de negocio, y cuenta con las propiedades ACID.

Gestor de transacciones Controla y supervisa la ejecución de transacciones.

7.1.2. Clasificaciones

- **Por conjuntos de instrucciones y datos (taxonomía de Flynn)**
 - La arquitectura **secuencial** es **SISD: Single Instruction Single Data.**

- Las arquitecturas paralelas son
 - **SIMD** Single Instruction Multiple Data.
 - **MISD** Multiple Instruction Single Data.
 - **MIMD** Multiple Instruction Multiple Data.
 - **SPMD** Single Program Multiple Data.
- **Por comunicación y control**
 - **Sistemas de multiprocesamiento simétrico (SMP)** Son MIMD con **memoria compartida** a través de la cual se comunican los procesadores.
 - **Sistemas MIMD con memoria distribuida** La memoria está **distribuida entre los nodos del sistema**, cada uno con su propia memoria local. Una red de interconexión conecta los procesadores mediante enlaces de comunicación. Los procesadores intercambian datos entre sus memorias **cuando se pide el valor de variables remotas**.

7.2. Tipos de paralelismo

- **A nivel de bit** Incrementando el **tamaño de la palabra** del microprocesador.
- **A nivel de instrucciones** Introduciendo **pipeline** en la ejecución de instrucciones máquina.
- **A nivel de búcle** Dividir las **interacciones de un búcle en partes** que se pueden realizar de manera complementaria.
- **A nivel de procedimientos** Identificando fragmentos de código de un programa que **pueden ejecutarse de manera simultánea sin interferir** la tarea de una a otra.
- **A nivel de tareas** Tareas que **cooperan para la solución del programa** general.
- **A nivel de aplicación** Conceptos de la **gestión de procesos** por parte del sistema operativo.

Granularidad (tamaño de grano) Medida de la **cantidad de computación** de un proceso software.

- **Paralelismo de grado fino** La paralelización se obtiene de forma casi automática.
- **Paralelismo de grano grueso** Paralelización de alto nivel que engloba al grano fino. **Requiere mayor conocimiento del código**, puesto que se paraleliza mayor cantidad de él.

7.3. Modelos de infraestructura para programación distribuida

- **Uso de sockets** Facilitan la **generación dinámica** de canales de comunicación. **No son adecuados** a nivel de aplicación.
- **Remote Procedure Call** Nivel de **invocación de procedimientos**. Adecuada para programación estructurada **basada en librerías**.
- **Invocación remota de objetos** Invocación de objetos que se encuentran **distribuidos por el sistema, localizados por identidad**. Adecuada para aplicaciones **orientadas a objetos**.
- **Remote Method Invocation** Solución Java para la **comunicación de objetos Java distribuidos**. Implica tiempo para hacer la serialización, enviar los objetos a través de la red y recomponerlos en el destino.
- **CORBA** Aplicaciones basadas en el **paradigma Cliente/Servidor**. Define servidores estandarizados a través de un modelo de referencia, los patrones de interacción y las especificaciones de las APIs.
- **MPI** Estándar que define la **sintaxis y la semántica** de las funciones contenidas en una biblioteca de paso de mensajes.